

HW1:

What is the output of the following code?

```
#include <iostream>

class Base {
    virtual void method() { std::cout << "from Base" << std::endl; }
public:
    virtual ~Base() { method(); }
    void baseMethod() { method(); }
};

class A : public Base {
    void method() { std::cout << "from A" << std::endl; }
public:
    ~A() { method(); }
};

int main(void) {
    Base* base = new A();
    base->baseMethod();
    delete base;
    return 0;
}
```

Ans:

from A

from A

from Base

HW2:

What is the output of the program!! And please explain what is going on?

```

#include <iostream>

class Base
{
public:
    virtual void foo() const { std::cout << "A's foo!" << std::endl; }
};

class Derived : public Base
{
public:
    void foo() { std::cout << "B's foo!" << std::endl; }
};

int main()
{
    Base* o1 = new Base();
    Base* o2 = new Derived();
    Derived* o3 = new Derived();

    o1->foo();
    o2->foo();
    o3->foo();
}

```

Ans:

A's foo!

A's foo! -> Because Base Class is **void foo() const** and Derived Class is **void foo()**, they aren't same signature. As a result, it will not be override.

B's foo!

HW3:

請以多型完成下列程式碼，演奏弦樂四重奏。

```

public class V {
    public static void main(String[] args) {
        // 弦樂四重奏
        Instrument[] stringQuartet = { new Violin(), new Violin(), new Viola(), new
Cello() };
        // play the music for me
        // implement HERE !!!
        For(Instrument instrument : stringQuartet){
            instrument.play();
        }
    }
}

abstract class Instrument {
    abstract public void play();
}

```

```
}
```

```
// 小提琴
```

```
class Violin extends Instrument {  
    @Override  
    public void play() {  
        System.out.println("旋律");  
    }  
}
```

```
// 中提琴
```

```
class Viola extends Instrument {  
    @Override  
    public void play() {  
        System.out.println("合旋");  
    }  
}
```

```
// 大提琴
```

```
class Cello extends Instrument {  
    @Override  
    public void play() {  
        System.out.println("低音");  
    }  
}
```

HW4:

Please implement (use polymorphism) the following code to make result generate the same output.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
class Base {
```

```
public:
```

```
    virtual void print() { cout << "Base" << endl; }
```

```
    virtual ~Base() {}
```

```
};
```

```
class A : public Base{
```

```
public:
```

```
    void print() { cout << "A" << endl; }
```

```
}
```

```
class B : public Base{
```

```
public:
```

```
    void print() { cout << "B" << endl; }
```

```
}
```

```
int main(void) {
```

```
    vector<Base*> bases = { new Base(), new A(), new B() };
```

```
    for (Base* b : bases) {
```

```
        b->print();
```

```
    }
```

```
    for (Base* b : bases) {
```

```
        delete b;
```

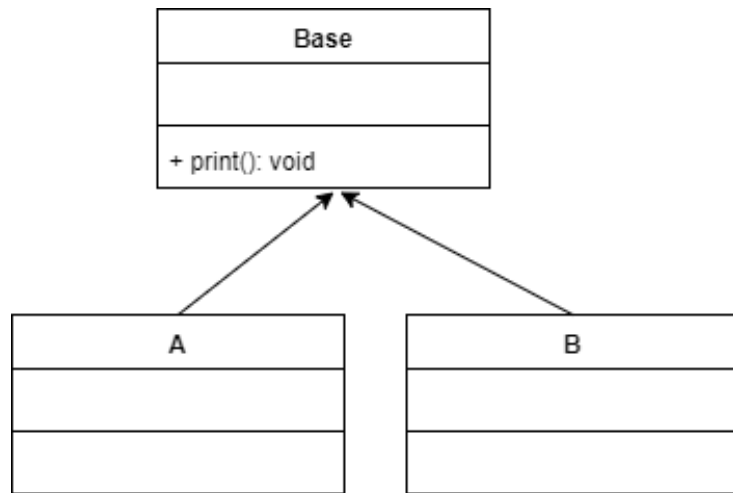
```
        b = nullptr;
```

```
    }
```

```
    bases.clear();
```

```
}
```

Please make inherit fit following class diagram.



Output:

```
$ ./a.exe
Base
A
B public Base
```

HW5:

Modify the attachment by using polymorphism.

```
#include <stdio.h>
#include <assert.h>

#define SIZE 3

typedef enum {
    Comic,
    Novel,
    Magazine
} book_type;

typedef struct Book {
    book_type type;
} Book;

void print_book_type(Book*);

int main(void) {
    Book books[SIZE];
    books[0].type = Comic;
    books[1].type = Novel;
    books[2].type = Magazine;

    for (int i = 0; i < SIZE; i++) {
        print_book_type(&books[i]);
    }

    return 0;
}

void print_book_type(Book* book) {
    assert(book);

    switch (book->type) {
    case Comic:
        printf("Comic\n");
        break;
    case Novel:
        printf("Novel\n");
        break;
    case Magazine:
        printf("Maganize\n");
        break;
    }
}
```

Ans:

```
#include <stdio.h>
#include <assert.h>
#define SIZE 3
typedef struct Book book;

struct book{
    void (*print_type)(Book*);
}

void print_comic(Book* book){
    printf("Comic\n");
}

void print_novel(Book* book){
    printf("Novel\n");
}

void print_magazine(Book* book){
    printf("Magazine\n");
}

void print_book_type(Book* book){
    assert(book);
    book -> print_type(book);
}

int main(void){
    Book books[SIZE];
    books[0].print_type = print_comic;
    books[1].print_type = print_novel;
    books[2].print_type = print_magazine;
    for(int i = 0; i < SIZE; i++){
        print_book_type(&books[i]);
    }
    return 0;}
```

HW6:

Please write a main function (with polymorphism way) to make result generate the same output.

```
#include <iostream>
using namespace std;

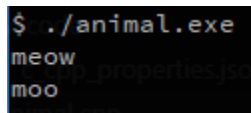
class Animal {
public:
    virtual void speak() = 0;
    virtual ~Animal() {}
};

class Cat : public Animal {
public:
    void speak() {
        cout << "meow" << endl;
    }
};

class Cow : public Animal {
public:
    void speak() {
        cout << "moo" << endl;
    }
};

int main(){
    Animal* animals[] = {new Cat(), new Cow()};
    for(Animal* animal : animals){
        animal->speak();
    }
    for(Animal* animal : animals){
        delete animal;
    }
    return 0;
}
```

Output:



```
$ ./animal.exe
meow
moo
```